ROUND 1 | THE PLAN

Baton

An evidence-linked, permission-aware knowledge handover platform

Problem statement: Prevent Critical Knowledge From Leaving With People - build a system that converts scattered project activity into a usable handover when a team member leaves.

Team name: ByteStorm

Submitted by: Ritika Bhati

Event: We Code Coders | Three-Round Challenge 01

Date: 5 November 2026

Core idea in one line: every machine-generated statement in a handover is a claim with a checked quote, a visible evidence label (Verified, Single-source, Conflicting, Missing, Human-asserted, Inferred) and an access label; human-added content is labelled as such, and nothing reaches the incoming person until a human approves it.

1. Understanding of the problem

When someone leaves a project, the files usually stay. What leaves is the ability to interpret them. A repository shows what the code does but not why a design was chosen over another. A task board shows tickets but not which ones are quietly blocked. A chat history holds a promise made to a vendor, buried deep in a long thread. The brief asks for a system that reconstructs the current state of the project from authorised sources and hands it over in a way a human can trust.

What exactly is lost

Type of knowledge	Where it hides today	What goes wrong after the exit
Decisions and their reasoning	Chat threads, meeting notes, PR comments	The successor re-opens settled debates or reverses a decision for the wrong reason
Unfinished work	Half-merged branches, stale tickets, personal to-do lists	Work is duplicated, or silently dropped
Dependencies and commitments	Emails, calls, verbal promises to vendors or mentors	Deadlines and obligations are missed because nobody knew they existed
Recurring problems and workarounds	The leaver's memory	The same failure is rediscovered, expensively
Who to ask	Informal relationships	The successor does not know which contact or reviewer to approach

Why existing documentation fails

- **Written once, at the wrong time.** A handover document is typically written in the last week, from memory, under time pressure.
- **It cannot be checked.** A narrative document has no links to evidence, so the reader cannot tell fact from recollection.
- **All-or-nothing on access.** Either the leaver shares everything (privacy and security risk) or nothing useful.
- **Generic AI summaries make this worse.** A fluent summary that invents an owner or hides a conflict looks authoritative and is more dangerous than no summary.

Which part we are solving (and which we are not)

We solve: turning scattered, authorised project activity into a reviewed, evidence-linked, role-appropriate continuation plan for one departing member and one incoming member of a project.

We deliberately do not solve: company-wide knowledge search, HR offboarding, or live monitoring of employees. Baton works on a project, with the consent of the people who own the sources, at the moment a handover is needed.

Evidence this affects real teams

Our argument starts from a pattern anyone who has joined a project midway will recognise: the files are there, the reasoning is not. This plan does not claim statistics or formal interviews. Our evidence is direct experience of taking over and handing off student-club and course projects, and the synthetic demo dataset deliberately encodes each of the failure patterns below so they can be tested repeatably.

2. Users affected

User	Role in a handover	Pain today	What Baton gives them
Incoming person	Takes over ownership	Long manual digging through old threads and files; fear of missing a hidden commitment	A continuation plan: owned items, risks, open questions, first-week actions, each with sources
Departing member	Knows the context	No time; documents from memory; worried about exposing private material	Auto-built draft they only correct; targeted questions instead of a blank page
Team lead / approver	Accountable for continuity	Cannot verify a handover; cannot tell what is missing	Review console showing evidence, conflicts and gaps; approve or reject per claim
Admin / security owner	Guards access	Fear that tools leak confidential chats or credentials	Source-level access labels, redaction, audit log, nothing shared unapproved

Departure and onboarding scenarios

Scenario	Environment	Typical sources
Student club office-bearer graduates	College club, annual leadership change	Shared drive docs, WhatsApp/Slack export, meeting notes, task sheet
Junior or intern developer finishes term	Small software team	GitHub issues, PRs, commits, README, standup notes
Research group member leaves	Lab or project group	Experiment logs, notebooks, meeting minutes, task tracker
Volunteer coordinator steps down	NGO or community event	Docs, contact sheet, chat export, task board

Primary persona we design and demo for: a team lead approving a handover from a departing developer or club member to a successor, because this single scenario exercises every step of the required workflow.

3. Proposed solution and what makes it different

Baton connects to a project's authorised sources and builds a **Claim Ledger**: a structured list of typed claims (decision, open question, owner, risk, dependency, next action; Round 3 adds blocker, recurring problem, contact). Every claim stores the exact supporting quote, a link to the source (with source ID

and version), its date and author, an evidence label and an access label. The handover document is simply a view of the approved ledger for a specific reader.

Evidence labels on every claim

Status	Meaning	How it is decided
VERIFIED	Quote exists in the cited source and is backed well enough (a machine check only; the reviewer's approval is the separate step that confirms the quote supports the claim)	Deterministic substring check passes AND the claim rests on one primary source (repo, tracked ticket, formal doc) or two independent informal sources (chat exports count from Round 3)
SINGLE-SOURCE	Plausible but rests on one informal mention	Quote check passes, but the only support is one informal source (chat or meeting notes)
CONFLICTING	Two sources disagree	Same topic key, different value (e.g. owner, date, chosen tool); both sides shown
MISSING	Expected knowledge has no evidence	Gap rules, e.g. an open task with no owner, a dependency with no contact, no deploy instructions found
HUMAN-ASSERTED	Added or changed by a person, not found in a source	Edits, departing-member answers and the first-week plan carry this label; never shown as VERIFIED
INFERRED	Model conclusion drawn from checked quotes that do not state it outright	Must list its supporting quotes (each verified in code); visible badge; needs reviewer confirmation before approval
QUOTE FAILED	Quote not found in the cited chunk	Kept out of the reader view; appears in the reviewer queue to be fixed or rejected

Note on Round 2. Chat exports are added only in Round 3, so claims backed by a single notes file or doc will usually show as SINGLE-SOURCE in Round 2. This is intended: Baton prefers an honest label over an inflated one.

What makes our approach different from a generic summariser

- **Claims, not paragraphs.** The unit of trust is a single claim with a quote, so a reviewer approves or rejects facts rather than proofreading prose.
- **Verification is code, not a model's opinion.** The model proposes claims; a deterministic verifier decides whether the quoted evidence really exists. Unverifiable claims are downgraded, never shown as fact.
- **Gaps are first-class output.** Baton reports what it could not find. Gap rules turn into targeted questions for the departing member ("Who owns the vendor contract mentioned in the 12 Oct notes?"). Their answers are stored as HUMAN-ASSERTED evidence, never confused with documents.
- **Permissions filter claims; they do not re-prompt the model.** The ledger is built once, then each role's view is produced by filtering claims against access labels. Filtering happens in code, not in a prompt, so it can be tested and guaranteed.
- **Acknowledgement closes the loop (Round 3).** The incoming person confirms receipt per section and can flag "this is wrong", which re-opens the claim for the approver.

Demo scenario (synthetic data): a club's rover project. The meeting notes say the team switched to motor driver B, but the latest commits still configure driver A. Baton marks the decision CONFLICTING, shows both quotes side by side, and asks the approver to resolve it before the handover can be published. This is exactly the failure a plain summary would hide.

4. Complete product workflow

Five steps: **1 Connect** (authorised sources) > **2 Reconstruct** (build the claim ledger) > **3 Verify** (evidence check) > **4 Review** (human in control) > **5 Handover** (role-based views and acknowledgement). Trust gates on every step: permissions inherited from sources; no machine claim without a checked quote; nothing shared until a human approves.

Step	What the user does	What the system does	Output
1. Connect	Team lead creates a handover case: names departing and incoming person, picks sources, sets access labels (team / leads only / private); only team-labelled sources are sent to the model.	Read-only ingestion of a GitHub repo and uploaded docs/notes (task CSV and chat exports join in Round 3). Secrets are stripped and names/emails replaced by pseudonyms before any model call, then restored from a local mapping table afterwards.	Indexed, labelled source chunks with stable IDs
2. Reconstruct	Waits a few minutes; sees progress per source.	LLM extracts typed claims in strict JSON, each with supporting quote(s) and chunk ID(s); builds a timeline of decisions, owners, dependencies.	Draft Claim Ledger
3. Verify	Reviews the dashboard of counts per evidence label.	Verifier matches every quote to its chunk, detects conflicts by topic key, runs gap rules, lists critical gaps (turned into questions for the leaver in Round 3).	Claims with evidence labels; gap list
4. Review	Approver edits, rejects, resolves conflicts and approves per claim; the departing member answers gap questions (Round 3).	Stores every edit as a new version with reviewer name and time. **Publishing rule:** every CONFLICTING claim must be resolved before publishing; critical MISSING gaps must be answered or explicitly accepted by the approver (the acceptance is logged and shown to the incoming person as a known gap).	Approved, versioned ledger
5. Handover	Incoming person opens a role-appropriate view, follows the first-week plan, clicks sources, acknowledges each section (Round 3).	Filters claims by the reader's permissions; shows restricted items as "hidden - ask approver"; logs access and acknowledgement.	Continuation plan + receipt

Round 2 delivers all five steps end to end with the Must features; the departing-member question flow and acknowledgement receipts are added in Round 3.

Incoming person's view (what they actually see)

- **Start here:** five-line project state and the three most urgent items.
- **Owned by you now:** tasks, systems, recurring duties, each with an evidence link.
- **Risks, blockers, dependencies:** ranked, with who to contact.
- **Decision timeline:** what was decided, when, why, and what is still disputed.
- **Open questions and known gaps:** clearly marked as unresolved.
- **First-week plan:** ordered next actions generated from open tasks, risks and deadlines, editable by the approver.

Illustrative screens (synthetic data, intended design - not a built product)

Review console | Rover club handover: A. Rao to N. Singh | draft v2 | Cannot publish: 1 conflict, 1 critical gap

Label	Item	Approver action
CONFLICTING	Decision: which motor driver? Meeting notes, 12 Oct: "we are moving to driver B" vs Commit a1b2c3, 30 Oct: "config: driver A pin map"	Keep driver B / Keep driver A / Reject both
VERIFIED	Owner: firmware is maintained by A. Rao. Quote (README.md, 2 Oct): "Firmware maintained by A. Rao"	Approve
MISSING	Critical gap: no deploy or run instructions found in any source	Add answer / Accept gap

Incoming person view | Handover for N. Singh | approved v3 by T. Lead | synthetic demo data

Label	Item	Source
VERIFIED	Flash firmware v2 to the rover before the 14 Nov demo	Issue #42 (GitHub), owner: you
SINGLE-SOURCE	Driver B vendor quote pending: contact S. Mehta	Meeting notes, 9 Oct (one informal source only)
RESTRICTED	Hidden for your role: ask the approver for access (placeholder only, no partial text)	-

5. Main features

Priority	Feature	Brief requirement it satisfies
Must (Round 2)	Connectors: GitHub repo (issues, PRs, commits) and Markdown/text/PDF upload for notes and docs	Import authorised data
Must (Round 2)	Claim extraction into six types (decision, owner, risk, dependency, open question, next action) with quote, source link, date and author	Reconstruct project state; citations
Must (Round 2)	Deterministic quote verifier, evidence labels, conflict and gap detection shown prominently	No assumptions as truth; highlight uncertainty and missing info
Must (Round 2)	Review console: edit, reject, approve per claim; simple version history	Human control, correction, approval
Must (Round 2)	Role-based views (approver, incoming person) with source-level access labels, secret stripping and pseudonymisation	Permission-aware sharing; protect confidential data
Should (Round 2)	Append-only audit-log table (screen in Round 3) of approvals, edits and views	Traceability
Should (Round 3)	Departing-member question flow (answers stored as HUMAN-ASSERTED evidence)	Capture tacit knowledge
Should (Round 3)	Task CSV and chat-export connectors (WhatsApp .txt, Slack JSON); extra claim types: blocker, recurring problem, contact	Richer sources and project state
Should (Round 3)	Decision timeline with diff between handover versions	Timeline, version history

Priority	Feature	Brief requirement it satisfies
Should (Round 3)	Acknowledgement receipts and incoming-person feedback loop	Acknowledgement of receipt
Could (Round 3)	Export of approved handover to PDF/Markdown	Usability

Scope discipline. We will build one complete journey end to end rather than many shallow integrations. Two source types in Round 2 (GitHub repo and uploaded notes/docs) are enough to demonstrate the whole workflow; task and chat imports arrive in Round 3. Live Slack/Gmail OAuth, mobile apps and enterprise SSO are explicitly out of scope for the event.

6. Technology stack, architecture and the hardest problem

Pipeline: authorised read-only connectors (GitHub repo; docs/PDF/MD notes; task CSV and chat exports in Round 3) > **ingestion** (parse, chunk with stable IDs and offsets, attach source ACL, strip secrets, pseudonymise names before any model call) > **extraction** (LLM to typed claims in a JSON schema) > **deterministic verifier** (quote match, conflict and gap rules) > **claim ledger** (PostgreSQL, versioned) > **review console** (edit, reject, approve, diff, history) > **role-based views** (filter claims, never re-prompt) > incoming person and acknowledgement.

Layer	Choice	Why this choice
Frontend	Next.js + TypeScript + Tailwind	Fast to build review and reader UIs; strong typing for the claim schema shared with the backend
Backend API	FastAPI (Python)	Best ecosystem for parsing, text processing and LLM SDKs; automatic OpenAPI docs help judges
Database	PostgreSQL (pgvector optional)	One store for claims, versions and ACLs. Access is enforced by application-level filtering covered by automated role tests; row-level security is a stretch goal
Background jobs	FastAPI background tasks (MVP)	Ingestion and extraction are slow; users should see progress, not a frozen page. Redis + RQ/Celery is a stretch goal
AI	Claude API with structured JSON output	Strict schema keeps extraction machine-checkable; the model proposes, never decides truth
Auth	OAuth login (GitHub/Google) + role table (approver, departing member, incoming person, admin)	Simple, realistic, and enough to demonstrate permission-aware views
Deployment	Docker Compose; Vercel + Render/Railway	Judges get a live link and a one-command local setup
Demo data	Synthetic rover-club project: repo, notes, tasks, chat export	Meets the synthetic-data rule; planted conflicts and gaps make tests repeatable

Most technically challenging part: trustworthy grounding

The hard problem is not generating a summary; it is guaranteeing that every statement is backed by real evidence and visible only to the right reader. Our pipeline never lets the model grade its own work:

- **Stable chunking.** Each source is split into chunks with permanent IDs and character offsets, so a citation can always be reopened at the exact passage.

- **Constrained extraction.** The model returns claims in a fixed JSON schema: type, topic key, value, basis (stated or inferred), and for each supporting quote: the quote, chunk ID, source ID and version, author and date. Inferred claims must list every quote they were drawn from. Anything outside the schema is discarded; an unknown author or date is stored as unknown, never guessed.
- **Deterministic verification.** Code normalises whitespace and checks that the quote is a substring of the exact (pseudonymised) text the model saw; the quote is then restored to real names from the mapping table and re-checked against the original chunk. Failure means the claim is downgraded and flagged, not shown as fact.
- **Conflict detection by topic key.** The model must pick each topic key from a per-project vocabulary that is lower-cased and alias-mapped (e.g. driver-b and DRV-B both map to motor-driver). Claims with the same key but different values are paired as CONFLICTING; only those pairs go to a model for a short explanation.
- **Gap rules.** A small, readable rule set creates MISSING claims (task without owner, dependency without contact, no run/deploy instructions). Gaps on owners, dependencies and run/deploy instructions are critical and block publishing until answered or explicitly accepted by the approver.
- **Access labels by inheritance.** A claim inherits the most restrictive label of its sources. Role views filter claims; restricted claims render as hidden placeholders, never as partial text.

Known risks and mitigations

- **Omitted claims:** gap rules and the question flow cover omissions, and the dashboard reports coverage honestly.
- **Slow large sources:** cap the demo corpus, process in background, show progress.
- **Prompt injection inside source text:** sources are treated as data, output is schema-only, quotes are verified in code. This limits, but does not eliminate, the risk.
- **Model failure or timeout:** ingested sources are kept, affected claims stay pending and are retried, and nothing partial is published. Extraction goes through a thin adapter so another model can be substituted.
- **Scope control:** PostgreSQL row-level security and Redis/RQ are stretch goals. The committed MVP uses FastAPI background tasks and application-level access filtering, covered by automated role-based tests (zero restricted claims visible to the incoming person). RLS is added in Round 3 only if time remains.
- **Pseudonymisation risk:** it can break exact-quote matching, so it is built and tested immediately after the verifier. If the restore-and-recheck step proves unreliable, the fallback is to send only team-labelled, non-personal text to the model and map names back by chunk offsets.

Model access to restricted data. After stripping and pseudonymisation, source text is sent to the Claude API for extraction. Sources labelled leads only or private are never sent to the model in the MVP; they can only be added by a human as HUMAN-ASSERTED claims. Pseudonyms are mapped back locally, so owners and contacts still appear correctly in the approved handover.

7. Privacy, security and trust design

Brief requirement	How Baton meets it
Do not present generated assumptions as verified truth	Evidence labels on every claim; claims whose quote fails stay out of the reader view until reviewed; assumptions carry a visible "inferred" badge
Do not expose confidential information merely because it is in a connected source	Connecting a source does not make it shareable: each source gets an access label, claims inherit it, and publishing needs approval
Allow authorised users to correct, reject and approve	Per-claim edit, reject and approve with reviewer identity and timestamp; full version history
Use synthetic, public, permissioned or anonymised data	All demos use synthetic data; connectors require explicit consent from the source owner; the upload screen warns against confidential material

Brief requirement	How Baton meets it
Secure data handling	Read-only scoped tokens; secrets stripped and identities pseudonymised before model calls (restored locally afterwards); leads-only and private sources never sent to the model; encrypted storage; delete-case option; audit log

8. Responsibilities of each team member

This is a solo submission, so one person owns every component. To keep the build feasible, Round 2 focuses on the five Must features and the core demo path; the Round 3 improvements are added only after that path is reliable. A short contribution log and the commit history will be kept from day one of the build window.

Member	Role	Owns	Round deliverables
Ritika Bhati	Solo - team lead, AI/verification, backend, frontend, product & QA	Extraction schema and verifier; conflict and gap rules; FastAPI, Postgres, ACL/role logic, connectors, redaction; review console, evidence viewer, incoming-person view; synthetic dataset, tests, docs and demo	R1: plan, evidence and dataset design. R2: five Must features end to end, deployment, docs. R3: question flow, timeline/diff, acknowledgement; hardening and tests

9. Development plan across three rounds

Round	Goal	What we will deliver	How we will know it works
Round 1: The Plan	Define the problem and a buildable design	This document, evidence from direct experience, synthetic dataset design, claim schema draft, wireframes (no product code before the build window)	Plan is specific enough to start building on day one
Round 2: The Build	A judge can run the core workflow end to end	The five Must features plus audit log, deployed link + repo, architecture doc, API/model disclosure	A judge uploads the demo project and reaches an approved handover; planted conflicts and gaps are detected
Round 3: The Finish	Complete, reliable, usable product	The three committed improvements plus task and chat connectors, testing notes, privacy/accessibility explanation, demo video, final report	Failure cases handled honestly; incoming person completes a handover with minimal help

Three improvements we commit to for Round 3

- **Departing-member question flow** that turns gaps into targeted questions and records answers as labelled evidence.
- **Decision timeline with version diff** showing how the project state and the handover changed between versions.
- **Acknowledgement and feedback loop** where the incoming person confirms or disputes each section and disputes re-open claims for review.

Build sequence inside the event (risk-first)

- **First third:** schema + chunking + verifier on the synthetic dataset, then pseudonymisation, because the trust model is the riskiest part.
- **Middle third:** extraction, conflict and gap rules; connectors for repo and uploads.
- **Final third:** review console, role views, deployment; test against the planted scenarios.
- **Buffer:** reserve at least 20% of time for fixes and the demo path. Cut list if time runs short: version diff, PDF export, audit-log screen (keep the table). The demo path (upload notes + connect repo, resolve the conflict, approve, view as incoming person) is never cut.

10. Expected impact

We state impact as targets we will measure in our own demonstration, not as claims about the world:

Measure	Target for the final demo	How we measure
Time to a reviewable handover draft	Minutes, not days, for the demo project	Timestamped runs on the synthetic dataset
Citation integrity	100% of displayed claims have a quote that passes the verifier; unverifiable claims are labelled	Automated test over all claims
Planted issues detected	All planted conflicts and at least 80% of planted gaps surfaced	Checklist against the dataset's answer key
Permission safety	Zero restricted claims visible in the incoming person's view	Role-based test cases
Usability	A first-time user reaches the first-week plan with minimal assistance	Observed test with 3-5 users

Beyond the event, the same approach aims to make handovers cheaper and less risky for student clubs, volunteer groups and small teams that lack formal knowledge-management tooling, and makes handovers auditable instead of dependent on one person's memory and goodwill.

11. Future scope

- Live connectors (Slack, Google Drive, Jira, Notion, email) using OAuth with per-source consent.
- Continuous mode: keep the ledger fresh so a handover is a review, not a reconstruction.
- Self-hosted and on-premise deployment with local models for sensitive organisations.
- Team-level templates for roles (e.g. club president, release manager) with role-specific gap rules.
- Enterprise features: SSO, retention policies, admin analytics, compliance exports.
- Evaluation suite for grounding quality that teams can run on their own data.

12. Complete team and participant details

Team name: ByteStorm

#	Full name (as on certificate)	Role	Email	Institution / organisation
1	Ritika Bhati	Solo - team lead / AI / backend / frontend / QA	ritika452044@gmail.com	Meerut Institute of Engineering and Technology

Disclosures

- **AI assistance: an AI assistant was used to help structure, draft and revise this plan; I reviewed and take responsibility for its content. Baton itself will use an LLM API (Claude via the Anthropic API) for claim extraction, disclosed in all later rounds.**
- **Reused material:** no product code exists for this event; open-source libraries and templates will be listed in the Round 2 document.
- **Data:** all demonstration data will be synthetic, public or anonymised; no confidential workplace material will be used.